

UPDATE

MCSL version 381p

PluginHive MCSL · Shopify / WooCommerce

| Story / card | Platform | Carrier scope | Toggle / prerequisite signal | Trello card link |

Generated June 2026 · PluginHive QA Team

Included Story Cards

Story / card	Platform	Carrier scope	Toggle / prerequisite signal	Trello card link
Release Label Batches	Shopify	Amazon Shipping	release.label.batch.enabled	https://trello.com/c/FSugx3SX
Delivro: Fall Back to BOITE when No Predefined Box is Selected	WooCommerce	FedEx	None detected	https://trello.com/c/Tuolk6lk
WSS: Add flat rate adjustment shown when carrier rates fail (ZD #394137)	WooCommerce	UPS	None detected	https://trello.com/c/NLqXsEv7
Shopify Billing Compliance and Trial Extension	Shopify	Carrier-neutral	None detected	https://trello.com/c/AH1WgQnW

How Support Should Use This Package

- › Use each card section as the support/demo guide for that feature.
- › Confirm customer/test platform, live store, carrier account, order/product, and toggle state before promising behavior.
- › Capture the escalation packet listed in the relevant card section if behavior does not match.

Release Label Batches - Release Label Batches

Support Guide: Release Label Batches

Release Label Batches — Label Batch page, Amazon Shipping / multi-carrier batch workflows

Trello card: <https://trello.com/c/FSugx3SX>

Feature Summary

Prior to MCSL 381p, merchants processing large label batches (1,000+ orders) could encounter a state where a batch became permanently stuck in **"Label Pending"** with no redirect, a misleading "Error in Batch Creation" message, and no self-service recovery path. All orders locked inside the stuck batch

were inaccessible for re-batching, and resolution required L3 intervention. This release introduces a **"Release Batch"** button on the Label Batch page, gated behind the account-level toggle `release.label.batch.enabled`. When triggered, it frees all orders in the selected stuck batch back to the order pool so the merchant can re-batch immediately — eliminating the L3 dependency for this class of issue.

Toggles & Prerequisites

Toggle / config note	What support should confirm
<code>release.label.batch.enabled</code> — account-level feature toggle	Confirm this toggle is enabled for the merchant's account before troubleshooting button visibility. If disabled, the "Release Batch" button will not appear under any condition.
MCSL version 381p	Confirm the store is running MCSL 381p or later. Earlier versions do not include this feature.
Shopify platform (primary test/customer platform)	Validate on Shopify Admin > Apps > PH Multi Carrier Shipping Label App. Do not assume another platform unless the ticket specifies one.
Amazon Shipping carrier account (context carrier)	Confirm the merchant has an active Amazon Shipping carrier configured under hamburger menu > Carriers if the stuck batch involves Amazon Shipping orders.
Label Batch page access	Confirm the merchant is navigating to the Label Batch page within the MCSL app — not the standard ORDERS tab single-label flow.
Batch must be in "Label Pending" state	The "Release Batch" button is only visible for batches in "Label Pending" state. Confirm the batch state before investigating button absence.
Multi-batch selection with mixed states	Pending QA sign-off — behaviour when multiple batches are selected and some are not in "Label Pending" state is marked as pending verification (□). Escalate if merchant reports unexpected button behaviour in mixed-selection scenarios.
Large batch release (1,000+ orders)	Pending QA sign-off — full release of batches with 1,000+ orders without partial failures is marked as pending verification (□). Monitor for partial-release reports on high-volume accounts.

Step-by-Step Support Walkthrough

1. Confirm the feature toggle is active for the merchant's account.

- › Check internally that `release.label.batch.enabled` is set to **true** for the account.
- › If the toggle is off, the "Release Batch" button will not appear regardless of batch state. Enable the toggle before proceeding with any further verification.

2. Confirm the merchant is on MCSL 381p or later.

- › In Shopify Admin > Apps > PH Multi Carrier Shipping Label App, check the app version indicator or confirm via internal release records.

3. Navigate to the Label Batch page in the MCSL app.

- › Path: Shopify Admin > Apps > PH Multi Carrier Shipping Label App > **Label Batch page** (accessible from the main navigation — confirm exact menu label from live store context if needed).
- › Identify any batches currently showing a **"Label Pending"** status.

4. Verify the "Release Batch" button appears for Label Pending batches.

- › Select one or more batches in "Label Pending" state.
- › Confirm the **"Release Batch"** button becomes visible in the batch action area.
- › If the button does not appear: re-confirm the toggle state (Step 1) and the batch status (must be "Label Pending", not "Success" or "Failure").

5. Verify the button does NOT appear for non-pending batches.

- › Select a batch in **Success** state — confirm "Release Batch" button is absent.
- › Select a batch in **Failure** state — confirm "Release Batch" button is absent.
- › This is a key guard rail; if the button appears for Success/Failure batches, treat as a regression and escalate.

6. Click "Release Batch" and observe the result.

- › After clicking, confirm:
- › The selected batch **disappears** from the Label Batch page.
- › All orders that were locked in the batch **return to the ORDERS tab order grid**.
- › Cross-check in Shopify Admin > Orders to confirm the orders are visible and no longer associated with the released batch.

7. Verify released orders can be re-batched immediately.

- › From the ORDERS tab, select the released orders.
- › Attempt to create a new label batch.
- › Confirm the orders are accepted into the new batch without error.
- › If orders remain locked or cannot be selected for a new batch, this is a regression — collect the order numbers and escalate.

8. For high-volume accounts (1,000+ orders in a batch), note the pending QA item.

- › Full release of 1,000+ order batches without partial failures is **not yet fully QA-verified** (□).
- › If a merchant reports that only some orders were released, collect the batch ID, order count, and any error messages, and escalate immediately — do not ask the merchant to retry without engineering review.

9. Check request/log evidence if the release action does not complete as expected.

- › Navigate to hamburger menu > **Settings** > **Request Log** (or equivalent log path in the MCSL app).
- › Filter by the relevant order or batch identifier and timestamp of the release attempt.
- › - Request/log fields to verify: batch ID, batch status at time of release action, order IDs returned to pool, any error codes or timeout indicators logged during the release call.

Expected Behaviour

What support should observe	How to confirm
"Release Batch" button appears when one or more "Label Pending" batches are selected	Select a "Label Pending" batch on the Label Batch page — button is visible in the action area
"Release Batch" button is absent for "Success" state batches	Select a batch in Success state — button does not appear
"Release Batch" button is absent for "Failure" state batches	Select a batch in Failure state — button does not appear
"Release Batch" button is absent when <code>release.label.batch.enabled</code> toggle is off	Disable toggle for test account — button does not appear for any batch state
Released batch disappears from the Label Batch page immediately after release	Label Batch page refreshes/updates — the released batch entry is no longer listed
All orders from the released batch return to the ORDERS tab order grid	Check ORDERS tab and cross-check Shopify Admin > Orders — orders are visible and selectable
Released orders can be added to a new batch without error	Select released orders in ORDERS tab, create new batch — batch creation succeeds
Orders locked in an unreleased "Label Pending" batch cannot be re-batched	Attempt to batch locked orders before releasing — system prevents re-batching (confirmed PASS in QA)
No partial release on standard batch sizes	All orders in the batch are freed — none remain locked after a successful release action
Large batch release (1,000+ orders) frees all orders without partial failures	☐ Pending full QA verification — monitor and escalate if partial releases are reported
Multi-stuck-batch selection triggers release correctly	☐ Pending full QA verification — escalate if button behaviour is inconsistent when multiple stuck batches are selected

Merchant-Safe Explanation

If you've ever had a label batch get stuck in "Label Pending" with no way to move forward, this update gives you a direct way to fix it yourself. A new **"Release Batch"** button now appears on your Label Batch page whenever you select a batch that's stuck in that state. Clicking it frees all the orders in that batch so they return to your order list, ready to be batched and labelled again straight away — no need to contact support to unlock them. The button only appears for stuck batches, so your completed or failed batches are not affected.

Common Questions & Troubleshooting

- › **"Release Batch" button is not visible at all.**
- › Check first: confirm the `release.label.batch.enabled` toggle is enabled for this account. This is the most common cause.
- › Also confirm the store is on MCSL 381p or later.
- › Confirm the merchant is selecting a batch in "Label Pending" state specifically — the button does not appear for Success or Failure batches by design.

- › **Button is visible but clicking it does nothing / shows an error.**
- › Inspect the Request Log (hamburger menu > Settings > Request Log) for errors at the time of the release attempt.
- › Note the batch ID and any error codes returned, and escalate to L3 with log evidence.
- › **Some orders returned to the grid but others are still locked after release.**
- › This is a potential partial-release scenario — a known pending QA item for large batches (1,000+ orders).
- › Do not ask the merchant to retry without engineering review. Collect batch ID, total order count, count of orders still locked, and escalate immediately.
- › **Merchant reports the batch disappeared but orders did not return to the ORDERS tab.**
- › Cross-check Shopify Admin > Orders to confirm whether the orders are visible at the Shopify level.
- › If orders are missing from both MCSL and Shopify Admin, escalate urgently with the batch ID and affected order numbers.
- › **Merchant wants to release a batch in "Failure" state using this button.**
- › Explain that the "Release Batch" button is intentionally limited to "Label Pending" batches only. Failure-state batch recovery may require a different process — confirm from live card/engineering context whether a separate flow exists for

Delivro: Fall Back to BOITE when No Predefined Box is Selected - Delivro: Fall Back to BOITE when No Predefined Box is Selected

Support Guide: Delivro – Fall Back to BOITE When No Predefined Box Is Selected

Delivro: Fall Back to BOITE when No Predefined Box is Selected

Trello card: <https://trello.com/c/Tuolk6lk>

Feature Summary

Prior to MCSL 381p, Delivro rate and label requests could fail when a merchant had no predefined Delivro box selected — for example, merchants using weight-based shipping, prepacked products, or self-packing configurations. This happened because MCSL was passing generic package types (YOUR_PACKAGING, CUSTOM_BOX) that Delivro's API does not recognise. Delivro only accepts its own parcel types: BOITE, ENVELOPPE, SAC, PALETTE, and AUTRE. This fix introduces a fallback: when no valid Delivro parcel type is present in the shipment context, MCSL now automatically substitutes BOITE before sending the request to Delivro. Merchants who have explicitly selected a Delivro-native parcel type continue to have that type passed through unchanged. Other carriers (e.g. CanPar, FedEx) are unaffected.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggle detected	Behaviour is active for all Delivro-enabled stores on MCSL 381p and above

Toggle / config note	What support should confirm
MCSL version prerequisite	Store must be on MCSL 381p or later — confirm app version in WordPress Admin / MCSL settings
Delivro carrier account configured	Confirm Delivro is added and active under hamburger menu > Carriers
Packaging method in use	Identify whether the merchant uses Weight-Based, Prepacked, Self Packing, or a Custom Box — fallback applies when none of these resolve to a Delivro-native parcel type
Delivro-native parcel type explicitly selected	If merchant has selected ENVELOPPE, SAC, PALETTE, or AUTRE on a product or box config, fallback must NOT apply — confirm pass-through is working
Other carriers on the same store (e.g. CanPar, FedEx)	Confirm those carriers still send YOUR_PACKAGING or their own package types without interference
Customer/test platform	WooCommerce (ADN Communications reference merchant: vapewolinak.com)

Step-by-Step Support Walkthrough

1. Confirm the MCSL app version on the WooCommerce store.

- › Navigate to WordPress Admin > Plugins (or the MCSL settings header) and verify the installed version is 381p or later.
- › If the store is on an earlier version, the fallback is not present — escalate for an update before continuing.

2. Confirm Delivro is active as a carrier.

- › In MCSL, go to hamburger menu > Carriers.
- › Verify Delivro appears in the carrier list and is enabled.
- › Confirm the Delivro account credentials are saved and the carrier is not in test/sandbox mode unless intentional.

3. Identify the merchant's packaging configuration.

- › Go to hamburger menu > Products (for prepacked or self-packing product-level box assignments) and hamburger menu > Settings > Packaging (for store-level box/packing method).
- › Determine which packaging method applies to the order under test — in each of the following scenarios the fallback applies and Delivro should receive `parcelType = B0ITE`:
 - › **Weight-Based** (no box assigned) → B0ITE
 - › **Prepacked** (`your_packaging` assigned) → B0ITE
 - › **Self Packing** (`your_packaging` assigned) → B0ITE
 - › **Custom Box** (`custom_box`) → B0ITE
- › In all these scenarios, the fallback should trigger and Delivro should receive `parcelType = B0ITE`.

4. Test rate retrieval for a Delivro shipment with no predefined Delivro box.

- › Open a WooCommerce order in WordPress Admin > WooCommerce > Orders, or use the MCSL ORDERS tab.
- › Trigger a rate request for the order using Delivro as the carrier.

- › Go to hamburger menu > Settings > Request Log (or the equivalent log viewer) and locate the outbound Delivro rate request.

- › - - Request node to verify: `parcelType` (or equivalent Delivro parcel type field in the rate request payload) — confirm value is `BOITE` when no Delivro-native type was configured.

5. Test label generation for the same order.

- › In the ORDERS tab, open the order and proceed to Prepare Shipment / Generate Label.

- › Select Delivro as the carrier and generate the label.

- › Inspect the shipment creation request in the request log.

- › - - Request node to verify: `parcelType` in the Delivro shipment/label creation request — confirm value is `BOITE`.

- › Confirm the label generates successfully (no API rejection error from Delivro).

6. Verify fallback applies across all packaging modes (QA-confirmed scenarios).

- › Repeat steps 4–5 for each of the following, checking the request log each time:

- › Weight-Based packaging (QA order ref: 27403 / label 1271)

- › Prepacked products (QA label ref: 1272)

- › Self Packing products (QA label ref: 1273)

- › Custom Box (QA label ref: 1274)

- › - - Request node to verify: `parcelType` = `BOITE` in each scenario where no Delivro-native type is present.

7. Verify multi-package shipments.

- › Create or locate an order that splits into multiple packages.

- › Generate a Delivro label and inspect the request log.

- › - - Request nodes to verify: `parcelType` on each package node in the multi-package request — all should show `BOITE` (or the explicitly selected Delivro type per package).

8. Verify that explicitly selected Delivro parcel types are passed through without fallback.

- › Assign a Delivro-native parcel type (e.g. `ENVELOPPE`, `SAC`, `PALETTE`, or `AUTRE`) to a product or box configuration under hamburger menu > Products or hamburger menu > Settings > Packaging.

- › Generate a rate and label for an order using that product/box.

- › Inspect the request log.

- › - - Request node to verify: `parcelType` — must reflect the explicitly selected type (e.g. `ENVELOPPE`), not `BOITE`. Fallback must not override a valid Delivro type.

9. Verify package edit functionality.

- › In the ORDERS tab, open an order, go to Prepare Shipment, and edit the package details (change parcel type or dimensions).

- › Re-trigger rate and label requests.

- › Inspect the request log to confirm the edited parcel type is reflected correctly (QA order ref: 27410).

- › - - Request node to verify: `parcelType` in both rate and label requests after the edit.

10. Verify no impact on other carriers (CanPar, FedEx).

- › Open an order and generate a rate/label using CanPar or FedEx on the same WooCommerce store.
- › Inspect the request log for those carriers.
- › - - Request node to verify: Package type field in CanPar/FedEx requests — must still show YOUR_PACKAGING or the carrier's own expected value, not B0ITE.

Expected Behaviour

What support should observe	How to confirm
Delivro rate request succeeds when no predefined box is selected	Request log shows a valid Delivro rate response; no API rejection error; parcelType = B0ITE in the outbound request
Delivro label generates successfully when no predefined box is selected	Label is created without error; request log parcelType = B0ITE in the shipment creation request
Fallback applies for Weight-Based, Prepacked, Self Packing, and Custom Box packaging modes	Each packaging mode produces a successful Delivro rate and label with parcelType = B0ITE in the request log
Multi-package Delivro shipments use B0ITE fallback on each package	All package nodes in the multi-package request log show parcelType = B0ITE (or the explicitly selected Delivro type)
Explicitly selected Delivro parcel types (ENVELOPPE, SAC, PALETTE, AUTRE) are passed through unchanged	Request log parcelType matches the selected type exactly; B0ITE fallback is not applied
Package edits are reflected correctly in subsequent rate and label requests	After editing package details in Prepare Shipment, the updated parcelType appears in the next rate and label request log entries
CanPar and FedEx requests are unaffected	Request logs for CanPar and FedEx continue to show their own package type values (e.g. YOUR_PACKAGING); no B0ITE value appears

Merchant-Safe Explanation

Delivro requires its own specific parcel type codes when requesting rates or generating labels — it does not accept the generic package formats used by other carriers. Previously, if you hadn't assigned a specific Delivro box type to your products or packaging settings, Delivro requests could fail silently or return errors. With this update, when no Delivro-specific box type is found, the system automatically uses Delivro's standard box type (B0ITE) so your rates and labels go through without any extra configuration on your end. If you have already set a specific Delivro parcel type (such as envelope or sack) on your products, that setting continues to be used exactly as before — nothing changes for those shipments.

WSS: Add flat rate adjustment shown when carrier rates fail (ZD #394137) - WSS: Add flat rate adjustment shown when carrier rates fail (ZD #394137)

Support Guide: WSS — Flat Rate Adjustment Suppressed When Carrier Rates Fail

WSS: Add flat rate adjustment shown when carrier rates fail (ZD #394137)

Feature Summary

Prior to MCSL 381p, when a carrier API (such as UPS) failed to return rates during checkout, any **Add Flat Rate** or **Add Flat Rate per Quantity** automation action configured alongside that carrier would incorrectly emit the flat rate adjustment as a standalone shipping option. This meant the merchant's customer was presented with — and could select — only the adjustment amount (e.g. \$5.00) rather than the intended carrier rate plus adjustment. The fix ensures that when a carrier is configured in the rule but returns no services (i.e. the API call failed), the flat rate adjustment is suppressed entirely and the carrier error is preserved. Flat-rate-only rules (where no carrier is attached) continue to emit the standalone flat rate as expected. This affects WooCommerce stores using Rate Automation rules with the **Add Flat Rate** or **Add Flat Rate per Quantity** action tied to a live carrier — UPS confirmed affected.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggle detected	Fix is code-level; no toggle needs to be enabled. Confirm store is on MCSL 381p or later.
MCSL version prerequisite	Store must be running MCSL version 381p . Confirm via WordPress Admin > Plugins or MCSL app version indicator.
Rate Automation rule using Add Flat Rate or Add Flat Rate per Quantity action	Confirm at least one active automation rule exists with this action type and a carrier (e.g. UPS) assigned.
UPS carrier account connected	Confirm UPS credentials are active under hamburger menu > Carriers. A failed API call is the trigger condition — credentials do not need to be broken, but the carrier must be configured in the rule.
WooCommerce platform	This fix was developed and tested on WooCommerce. Confirm the merchant is on a WooCommerce store.
Carrier API failure condition	The bug only surfaces when the carrier API fails to return rates at checkout. Confirm whether the merchant can reproduce a UPS API failure (e.g. unsupported destination, service outage, credential issue) to validate the fix.

Step-by-Step Support Walkthrough

1. Confirm MCSL version

Navigate to **WordPress Admin > Plugins** and locate the PluginHive Multi-Carrier Shipping Label plugin. Confirm the installed version is **381p** or later. If the store is on an earlier version, the fix is not present — escalate for update.

2. Locate the affected Rate Automation rule

In the MCSL app, go to **hamburger menu > Settings > Shipping Rates / Automation** (or the equivalent Rate Automation section on the WooCommerce store's MCSL instance). Identify any rule that uses the **Add Flat Rate** or **Add Flat Rate per Quantity** action with a carrier (UPS or otherwise)

assigned to the same rule.

3. Confirm UPS carrier configuration

Go to **hamburger menu > Carriers** and confirm UPS credentials are present and the carrier is active. Note the carrier account details for the escalation packet if needed.

4. Reproduce or simulate the carrier API failure scenario

Work with the merchant to identify a condition under which UPS returns no rates (e.g. an unsupported destination postcode, a service not available for the shipment dimensions, or a known credential/API issue). Place a test order or simulate checkout with those parameters on the **WooCommerce storefront**.

5. Observe checkout rate behaviour

At the WooCommerce checkout, check which shipping options are displayed when UPS fails to return rates:

- › **Before fix (incorrect):** The flat rate adjustment amount appears as a standalone shipping option (e.g. "\$5.00 — Flat Rate").
- › **After fix (correct):** No shipping rate is shown for that rule; the carrier error is preserved and the flat rate adjustment is suppressed.

6. Verify via Request Log

In the MCSL app, go to **hamburger menu > Settings > Request Log** (or the equivalent log viewer). Pull the log entry corresponding to the failed checkout rate request.

- › **Request/log fields to verify:** Confirm that `carrierId` is present in the automation rule's applied actions object, that `services` is empty (indicating the carrier API returned no rates), and that no flat rate entry is emitted in the response payload for that rule. If `carrierId` is undefined (flat-rate-only rule), a standalone flat rate entry should still appear — confirm this is not being suppressed.

7. Validate flat-rate-only rules are unaffected

If the merchant has any automation rules using **Add Flat Rate** with **no carrier assigned** (pure flat-rate rules), confirm those still display the flat rate at checkout as expected. This is the intended carve-out in the fix.

8. Cross-check WooCommerce order data

If the merchant reports a past order where the customer was undercharged, go to **WordPress Admin > WooCommerce > Orders**, locate the affected order, and confirm the shipping amount charged. This helps establish whether the bug was triggered pre-fix and supports any refund/adjustment conversation with the merchant.

Expected Behaviour

What support should observe	How to confirm
When UPS (or any carrier) fails to return rates and a carrier is assigned to the rule, no flat rate adjustment is shown at WooCommerce checkout	Place a test order with a UPS-failure condition; confirm no standalone flat rate option appears for that rule

What support should observe	How to confirm
The carrier error/failure state is preserved and surfaced (no phantom rate replaces it)	Check Request Log — the response for the failed carrier rule should contain no emitted rate entry
Flat-rate-only rules (no <code>carrierId</code> assigned) continue to display the standalone flat rate at checkout	Test a rule with no carrier assigned; confirm flat rate still appears normally
No undercharging scenario: customers cannot select a rate equal only to the adjustment amount when a carrier is configured	Review WooCommerce checkout rate options during a UPS failure — only valid carrier rates (or no rates) should appear
<code>carrierId</code> present + services empty → flat rate suppressed	Confirm in Request Log: <code>carrierId</code> set, <code>services</code> array empty, no flat rate node in output
<code>carrierId</code> absent (undefined) + flat-rate rule → flat rate emitted	Confirm in Request Log: <code>carrierId</code> undefined, flat rate node present in output

Merchant-Safe Explanation

Previously, if UPS was unable to return shipping rates at checkout — for example, due to a temporary API issue or an unsupported destination — your store may have displayed only the flat rate adjustment amount (such as \$5.00) as a shipping option, rather than the full carrier rate. This could result in customers being charged far less than the actual shipping cost. This has now been corrected: when UPS fails to return rates, that shipping option is hidden entirely at checkout, so customers are never presented with an incomplete or incorrect rate. Any flat rate rules you have set up that are not tied to a carrier will continue to work exactly as before.

Common Questions & Troubleshooting

- › **Check first:** Confirm the store is on MCSL **381p** or later. If not, the fix is not active and the bug will still occur — the store needs to be updated before any further troubleshooting.
- › **Check first:** Confirm the affected automation rule has a carrier (UPS) assigned. If the rule is a flat-rate-only rule with no carrier, the standalone flat rate displaying is correct behaviour, not a bug.
- › **If the flat rate is still appearing after the update:** Verify the rule configuration in **hamburger menu > Settings > Shipping Rates / Automation** — confirm `carrierId` is actually set on the rule. If the rule was built without a carrier, the fix intentionally allows the flat rate through.
- › **If no rates appear at all (including expected flat-rate-only rules):** Check whether the flat-rate-only rule has inadvertently had a carrier assigned. Review the rule configuration and Request Log output.
- › **Inspect logs when:** The merchant reports customers are still being undercharged, or when a flat rate is appearing unexpectedly during a carrier failure. Pull the Request Log entry for the checkout session and verify the `carrierId` and `services` fields in the automation actions output.
- › **Inspect logs when:** The merchant reports that a flat-rate-only rule has stopped working after the update. Confirm `carrierId` is absent on that rule in the log output.

- › **Escalate when:** The fix appears to be applied (381p confirmed) but the incorrect flat rate is still surfacing during a carrier failure — this may indicate a rule configuration edge case or a regression. Include the full Request Log entry and rule configuration screenshot in the escalation packet.
- › **Escalate when:** The merchant has confirmed customer orders were placed at the undercharged flat rate amount and is requesting a resolution — this is a billing/financial impact escalation.
- › **Do not assume** the bug affects all carriers. UPS is the confirmed affected carrier. If the merchant reports the same behaviour with a different carrier, note it but do not confirm scope without escalation.

Support Escalation Packet

- › **Story card:** WSS: Add flat rate adjustment shown when carrier rates fail (ZD #394137)

Trello URL: <https://trello.com/c/NLqXsEv7/4575-wss-add-flat-rate-adjustment-shown-when-carrier-rates-fail-zd-394137>

- › **ZD ticket:** <https://pluginhive.zendesk.com/agent/tickets/394137>
- › **Store URL:** Confirm from live store/card context
- › **Account UUID:** Confirm from live store/card context
- › **Carrier account:** UPS — confirm account ID / credentials reference from hamburger menu > Carriers
- › **Affected order number(s):** Confirm from WooCommerce Admin > Orders (orders where shipping charged = flat rate adjustment amount only)
- › **MCSL version on store:** Confirm from WordPress Admin > Plugins

Shopify Billing Compliance and Trial Extension - Shopify Billing Compliance and Trial Extension

Support Guide: MCSL 381p — Shopify Billing Compliance and Trial Extension

Shopify Billing Compliance and Trial Extension

Trello card: <https://trello.com/c/AH1WgQnW>

Feature Summary

This release addresses a Shopify billing compliance flag raised against the PH Multi Carrier Shipping Label app for allowing merchants to activate the same subscription plan multiple times. The fix introduces a server-side check against Shopify's existing active charges before any new charge is created, and disables re-selection of the currently active plan in the billing UI. Separately, the same release adds a proper trial extension mechanism: admins can now extend merchant trials directly from the admin panel without requiring a reinstall. If a trial is still active, the extension applies silently; if the trial has already expired, the merchant sees a one-click **Extend Trial** prompt on their next app visit. Both changes are Shopify-specific and carrier-neutral.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggles detected	Confirm the store is on MCSL version 381p or later before troubleshooting billing or trial behaviour
Shopify billing active-charge check (server-side)	Confirm the store has an active or previously active Shopify subscription charge visible in Shopify Admin > Apps > PH Multi Carrier Shipping Label App
Trial extension (admin-initiated)	Confirm the admin has used the storepep-react admin panel to extend the trial; this is not a merchant-facing setting
Trial extension prompt (expired trial)	Confirm the merchant's trial has expired and they have not yet selected a paid plan; the prompt should appear on next app visit
Trial extension prompt suppression	Confirm the merchant previously had a trial — the "Extend Trial" prompt must NOT appear for merchants who never had a trial
Same-plan re-selection block (UI)	Confirm the merchant's current active plan is visually disabled/unselectable in the billing plan selection screen
Same-plan re-activation after merchant-cancelled charge	Confirm the merchant cancelled the charge directly via Shopify Admin (not via the app); re-activation should be permitted in this case
Test stores referenced in QA	raj-trail.myshopify.com, mcsL-loggedin354.myshopify.com, mcsL-ashok.myshopify.com — use these for internal reproduction; confirm from live store context for merchant tickets

Step-by-Step Support Walkthrough

Scenario A — Verifying same-plan re-selection is blocked

1. Ask the merchant to open **Shopify Admin > Apps > PH Multi Carrier Shipping Label App** and navigate to the billing/plan selection screen.
2. Identify the merchant's currently active plan. Confirm the plan tile or button for that plan is visually disabled or non-interactive in the UI.
3. If the merchant reports they were able to re-select the same plan, ask whether they attempted this through the app UI or via a direct API call or URL manipulation.
 - › If via UI: confirm the store is on MCSL 381p. If not, this is a version issue — escalate.
 - › If via direct API bypass: the server-side charge check should still block duplicate charge creation. Ask the merchant to share the error or outcome they received.
4. To verify the server-side block is active, check **request/response logs** for the billing charge creation endpoint.
 - › - Request/response nodes to verify: existing active charge ID returned by Shopify charge lookup, HTTP response status on duplicate charge attempt (expect rejection/no new charge created)
5. If the merchant cancelled their charge directly through **Shopify Admin > Apps > Manage subscriptions** (not through the app), re-activation of the same plan should be permitted. Confirm this is the merchant's scenario before treating it as a bug.

Scenario B — Verifying trial extension (active trial)

1. Confirm the merchant's trial is currently active by checking **Shopify Admin > Apps > PH Multi Carrier Shipping Label App** — the trial end date should be visible.
 2. Confirm the admin has initiated the trial extension from the **storepep-react admin panel**. This action is not available to merchants directly.
 3. After the admin extends the trial, the merchant should see no interruption or prompt — the extension applies silently.
 4. Ask the merchant (or verify on the test store) that the trial end date has moved forward as expected.
- › - Request/log fields to verify: updated trial end date/timestamp in the billing charge record returned by Shopify, admin action log entry in storepep-react
5. Confirm the merchant was not asked to reinstall the app as part of this process. If reinstall was required, this is unexpected behaviour — escalate.

Scenario C — Verifying trial extension prompt (expired trial)

1. Confirm the merchant's trial has expired and they have not yet activated a paid plan.
 2. Ask the merchant to open **Shopify Admin > Apps > PH Multi Carrier Shipping Label App** (their next app visit after expiry).
 3. The merchant should see a one-click **"Extend Trial"** prompt on the app landing/billing screen.
 4. Confirm the merchant can click the prompt and the trial is extended without requiring reinstall or plan selection.
 5. Confirm the **"Extend Trial"** prompt does NOT appear for merchants who never had a trial period on their account.
- › - Request/log fields to verify: trial eligibility flag or original trial record in the merchant's billing history; absence of trial record should suppress the prompt

Scenario D — Verifying plan upgrade and downgrade behaviour

1. Ask the merchant to select a higher-tier plan from the billing screen in **Shopify Admin > Apps > PH Multi Carrier Shipping Label App**.
2. Confirm the upgrade completes successfully and a new Shopify charge is created (the existing active charge check should not block a genuine plan change).
3. For downgrade: confirm the plan change completes. Per QA notes, changing plan will not sustain the trial period — inform the merchant of this if they ask why their trial days did not carry over after a plan change.

Scenario E — Verifying uninstall and reinstall behaviour

1. If the merchant uninstalls and reinstalls the app, confirm the billing flow starts fresh — a new charge creation attempt should be permitted.

2. Confirm the same-plan block does not incorrectly prevent charge creation on a clean reinstall.
 3. If the merchant had an active or expired trial at the time of uninstall and reinstalls, confirm the trial extension flow behaves correctly per Scenarios B and C above.
- › - Request/log fields to verify: charge status at time of reinstall (should be cancelled/expired, not active), new charge creation response from Shopify

Expected Behaviour

What support should observe	How to confirm
Currently active plan tile is disabled/unselectable in the billing UI	Open billing/plan screen in Shopify Admin > Apps > PH Multi Carrier Shipping Label App; active plan button is non-interactive
Duplicate charge creation is blocked server-side even if UI is bypassed	Check request/response logs for billing endpoint; no new charge ID created when same plan is re-submitted while an active charge exists
Trial extension applied silently when trial is still active	Admin extends trial via storepep-react admin panel; merchant sees no prompt or interruption; trial end date moves forward
"Extend Trial" one-click prompt appears on merchant's next visit after trial expiry	Merchant opens app after trial expiry; prompt is visible on billing/landing screen
"Extend Trial" prompt does NOT appear for merchants who never had a trial	Open app on a store with no trial history; no prompt is shown
Plan upgrade creates a new charge correctly	Select higher-tier plan; new Shopify charge is created and confirmed active
Plan downgrade works; trial period does not carry over	Select lower-tier plan; plan changes successfully; trial days are not preserved post-change
Reinstall after uninstall allows fresh charge creation	Reinstall app; billing flow initiates normally without being blocked by the existing-charge check
Same-plan re-activation permitted after merchant cancels charge via Shopify Admin	Merchant cancels charge in Shopify Admin > Apps > Manage subscriptions; re-selecting same plan in app succeeds
Repeated trial extensions on an active trial succeed	Admin extends trial multiple times in a row; each extension updates the trial end date correctly

Merchant-Safe Explanation

We've made improvements to how the app handles your subscription plan and trial period on Shopify. You'll notice that your current active plan can no longer be accidentally re-selected, which prevents duplicate charges from being created. If you're on a trial and need more time, your account manager can extend your trial directly — you won't need to uninstall and reinstall the app. If your trial has already ended, you'll see a simple one-click option to extend it the next time you open the app. These changes ensure your billing stays accurate and your trial experience is smoother.

Common Questions & Troubleshooting

- › **Merchant says they can't select their current plan:** This is expected behaviour as of MCSL 381p. The active plan is intentionally disabled to prevent duplicate charges. If they want to change plans, they should select a different tier.
- › **Merchant says the "Extend Trial" prompt is not appearing after their trial expired:** First confirm the store is on MCSL 381p. Then confirm the merchant originally had a trial — the prompt only appears for merchants with a prior trial record. If both are confirmed and the prompt is still missing, escalate with store URL and billing history.
- › **Merchant says the "Extend Trial" prompt is appearing but they never had a trial:** This is unexpected. Escalate immediately with store URL, account UUID, and a screenshot of the prompt.
- › **Merchant reports a duplicate charge was created:** Check request/response logs for the billing charge creation endpoint to confirm whether the server-side active-charge check ran. Confirm the store is on 381p. If a duplicate charge was created on 381p, escalate with full log evidence.
- › **Admin trial extension is not working (restart subscription issue):** QA noted an issue with "Restart subscription" from the admin view on `raj-trail.mysh`